

Plan y reporte de pruebas — Ecosistema BabyWolf

Materia: Desarrollo para Dispositivos Inteligentes · Evaluación 2 · Mayo–Agosto 2026

Todas las pruebas se ejecutaron sobre el sistema completo: hub en la laptop, `Pixel_9` y `Wear_OS_Large_Round` en emulador, y la PWA en Chrome a 1920×1080. Los resultados de esta tabla son **medidos, no estimados**; las evidencias están en [evidencias/](#).

Resumen

Total	Pasan	Fallan
19	19	0

Casos de prueba

CP-01 · La API entrega noticias reales

Precondición: conexión a internet. **Pasos:** `curl https://babywolf-blog-production.up.railway.app/api/posts` **Esperado:** lista de noticias publicadas con título, slug, contenido y portada. **Resultado:** ✅ 6 noticias. `{"id":"...", "title":"El Renacer de los 8-bits", "slug":"renacer-8-bits", ...}`

CP-02 · La API entrega la categoría de cada noticia

Motivo: el wearable agrupa por tema; sin `category` no hay temas que mostrar. **Pasos:** `curl .../api/posts | grep -o '"category": "[^"]*"' | sort | uniq -c` **Esperado:** cada noticia trae su categoría. **Resultado:** ✅ 2 retro, 2 tech, 1 gaming, 1 opinion.

CP-03 · El códec GATT conserva los tres tipos de dato

Motivo: los bytes no llevan tipo; si el códec falla, las métricas llegan corruptas. **Pasos:** `cd wearable_app && flutter test` **Esperado:** ida y vuelta exacta para string, uint16 y float32; valor truncado rechazado. **Resultado:** ✅ 5/5 pruebas pasan. Incluye recorte fuera de rango (70000 → 65535) y `FormatException` ante bytes insuficientes.

CP-04 · El teléfono muestra las noticias reales (P2.5)

Pasos: abrir la app con red disponible. **Esperado:** feed con portada, categoría, fecha, título y extracto. **Resultado:** ✅ Ver `03-telefono-feed.png`.

CP-05 · El teléfono maneja el error de red (P2.5)

Pasos: `adb shell svc wifi disable && svc data disable`, reiniciar la app. **Esperado:** mensaje claro y opción de reintentar; **la app no se cierra**. **Resultado:**  Pantalla “No se pudieron cargar las noticias” con el detalle del error y botón Reintentar. Proceso vivo (`pid 8168`). Al restaurar la red, Reintentar recarga el feed. Ver `08-telefono-error-de-red.png`.

CP-06 · El wearable genera datos cada segundo

Pasos: abrir la app en `Wear_05_Large_Round` y pulsar **Iniciar**. **Esperado:** los tres valores cambian una vez por segundo y el tema rota. **Resultado:**  El botón pasa a **Detener**, el icono de consola toma el glow y los contadores avanzan cada segundo. Ver `02-wearable-generando.png`.

CP-06b · Los temas y el contador del wearable salen de la API, no de un `Random`

Motivo: un dato inventado no demuestra nada. Los temas del reloj tienen que ser las categorías reales del blog y el contador, el número real de noticias. **Pasos:** comparar lo que muestra el reloj con `GET /api/posts`. **Esperado:** coincidencia exacta en total, categorías y orden.

	La API dice	El reloj muestra
Total de noticias	6	<code>sin leer de 6</code>
Categorías	retro 2, tech 2, opinion 1, gaming 1	4 iconos de consola
Orden por volumen	retro → tech → opinion → gaming	mismo orden en la tira

Resultado:  Coincidencia exacta. El indicador **API** del reloj se pone verde cuando la consulta responde. La app vuelve a preguntar cada 60 s: si se publica una noticia durante la demo, el contador sube solo.

Lo único simulado es el acto de leer —una noticia cada 30 s—, porque es actividad del usuario y ningún emulador tiene un sensor que la mida. El cronómetro de minutos sí es real. Si la API no responde, el reloj sigue funcionando con el último dato bueno y el indicador API se apaga.

CP-07 · Los datos del wearable llegan al teléfono por NOTIFY (P2.6)

Pasos: con ambos emuladores activos, abrir el panel del wearable en el teléfono. **Esperado:** las tres métricas se actualizan en vivo. **Resultado:**  Tema activo: `OPINION` (string), Noticias sin leer: `6` (uint16), Minutos de lectura: `0.2` (float32), **273 notificaciones GATT recibidas**. Los tres tipos se decodifican correctamente. Ver `04-telefono-wearable-conectado-alerta.png`.

CP-08 · El umbral crítico dispara una alerta visible

Umbral: `noticias sin leer > 4`. Se fija en 4 porque el blog ronda las 6 noticias publicadas: alto para que no salte por cualquier cosa, bajo para que se pueda apagar leyendo un par durante la demo. **Esperado:** aviso visible sin abrir ningún menú, y que se apague solo al bajar del umbral. **Resultado:**  Con las 6

noticias sin leer aparecen **dos** avisos: banner rojo sobre el feed (“Backlog crítico: 6 noticias sin leer en opinion”) y bloque de alerta en el panel. El indicador del reloj en la barra superior pasa a rojo.

Se comprobó también el ciclo completo: conforme el reloj va leyendo, el contador baja (6 → 3 → 1) y **la alerta se apaga sola** al cruzar el umbral, sin recargar nada.

CP-09 · Desconectar el wearable no rompe el teléfono

Pasos: `adb shell am force-stop com.babywolf.wearable_app`, esperar el timeout. **Esperado:** estado “desconectado”, sin cierre inesperado. **Resultado:**  El panel muestra “**Wearable desconectado**” con indicador gris y conserva los últimos valores recibidos: `OPINION`, 5 sin leer, 0.9 min y 156 notificaciones GATT. La alerta de umbral sigue visible porque el último dato la superaba. Proceso vivo (`pid 3169`). Ver `06-telefono-wearable-desconectado.png`.

CP-10 · La PWA cabe en 1080 px sin scroll

Esperado: `scrollWidth` y `scrollHeight` exactamente 1920×1080. **Resultado:**  `{bodyScrollW: 1920, bodyScrollH: 1080}`. Ninguna dimensión desborda.

CP-11 · Navegación D-pad en las cuatro direcciones

Pasos: recorrer las 4 direcciones desde cada una de las 4 tarjetas (16 combinaciones). **Esperado:** 8 movimientos válidos y 8 bordes donde el foco se queda quieto. **Resultado:**  16/16 correctos.

Desde	↑	↓	←	→
card0	borde	card2	borde	card1
card1	borde	card3	card0	borde
card2	card0	borde	borde	card3
card3	card1	borde	card2	borde

En los 8 bordes el foco **no se pierde ni salta**: permanece en la tarjeta actual.

CP-12 · Enter/OK cambia el recurso multimedia de fondo

Pasos: enfocar una tarjeta y pulsar Enter. **Esperado:** el fondo pasa a la portada de esa noticia. **Resultado:**  `card0` → `supabase.co/storage/...`, `card3` → `images.unsplash.com/photo-1542751371...`. El titular grande también se actualiza.

CP-13 · Fallback si la portada no carga

Pasos: `ponerFondo('https://images.unsplash.com/no-existe-esta-imagen.jpg')` **Esperado:** no queda una imagen rota; se ve el color sólido. **Resultado:**  `backgroundImage = none`, queda el fondo `#1a1a2e`. La pantalla nunca se ve rota.

CP-14 · Modo offline con service worker

Pasos: cargar la PWA, **apagar el servidor** (`lsof -ti:3000 | xargs kill`) y recargar. **Esperado:** la app sigue cargando desde cache. **Resultado:** ✅ Con el servidor caído, la PWA carga completa: header, titular, grid 2x2 y footer. 10 estáticos en `babywolf-tv-static-v1` y la cache dinámica creada para la API.

CP-15 · Sincronización teléfono → TV en menos de 2 segundos

Pasos: abrir una noticia en el teléfono y cronometrar hasta que la TV la refleje. **Esperado:** < 2000 ms.

Resultado: ✅ **995 ms, 997 ms y 1371 ms** en tres mediciones. Peor caso 1371 ms, dentro del límite. El sondeo es de 1 s, así que el techo teórico es ~1 s más la latencia local.

Verificado además de extremo a extremo: al tocar “Esta es una prueba” en el teléfono, el hub pasó de `renacer-8-bits` a `esta-es-una-prueba` y la TV cambió su titular a esa noticia.

CP-16 · El APK de release está firmado y es instalable

Pasos: `flutter build apk --release`, verificar con `apksigner` e instalar en el emulador. **Esperado:** firmado con el certificado propio (no el de debug), instalable y funcional. **Resultado:** ✅ APK de 46.5 MB.

```
$ apksigner verify --print-certs app-release.apk
V2 Signer: certificate DN: CN=Abraham Duran, OU=Uteq, O=Uteq, L=Queretaro, ST=Qro, C=MX
V2 Signer: certificate SHA-256 digest:
1d2bd136438f7e6fc13d5e4f99db12e709bb60b029b81f4333c2f1b29a8c6665

$ apksigner verify app-release.apk ; echo $?
0
```

Confirmación adicional de que **no** usa la firma de debug: al intentar instalarlo sobre el APK de debug, Android lo rechazó con `INSTALL_FAILED_UPDATE_INCOMPATIBLE: signatures do not match`. Tras desinstalar el debug, la instalación fue `Success` y la app arrancó cargando noticias reales (`pid 3748`). Ver `09-telefono-apk-release-firmado.png`.

CP-17 · Contraste WCAG AA en la pantalla de TV

Motivo: a tres metros de distancia, un texto con poco contraste deja de leerse. **Método:** cálculo del ratio de contraste según la fórmula de luminancia relativa de WCAG 2.1, tomando el **peor caso**: una portada completamente blanca detrás del degradado y de las tarjetas translúcidas. **Esperado:** todos los textos $\geq 4.5:1$. **Resultado:** ✅ tras corregir el color de texto.

Elemento	Tamaño	Contraste	AA (4.5:1)
Titular de la noticia	80 px	13.75:1	✓
Título de tarjeta	40 px	12.21:1	✓
Foco dorado sobre tarjeta	—	9.47:1	✓
Footer	24 px	7.46:1	✓
Fecha de tarjeta	24 px	6.87:1	✓
Etiqueta del hero	32 px	5.60:1	✓
Categoría de tarjeta	32 px	4.97:1	✓

Corrección aplicada. El `#e94560` de la guía de estilos daba **3.57:1** sobre las tarjetas. Cumple el mínimo de WCAG para texto grande (3:1), que es el que técnicamente le corresponde a un texto de 32 px, pero no el 4.5:1 general. Se introdujo `--neon-texto: #f2748a`, una derivación más clara del mismo tono, que se usa **sólo en texto**; el `#e94560` original se conserva en bordes, glows y botones, donde no hay requisito de legibilidad. La misma corrección se aplicó al chip de categoría del teléfono, que a 11 px sí está sujeto al 4.5:1 estricto.

CP-18 · Cada dispositivo funciona por su cuenta

Motivo: en una demo en vivo algo puede fallar. Conviene saber qué se cae con qué, y que ninguna pieza arrastre a las demás. **Pasos:** apagar componentes de uno en uno y observar el resto.

Se apaga	El wearable	El teléfono	La Smart TV
App del teléfono	✓ Sigue igual: GATT y API en verde, generando	—	✓ Sigue mostrando noticias; deja de recibir selecciones nuevas
Hub	✓ Sigue generando y consultando la API; marca GATT en gris	⚠ “Sin enlace: ¿está corriendo el hub?”; el feed de noticias sigue	✓ Sigue mostrando noticias; se pierde la sincronía
Red / API	✓ Sigue con el último dato bueno; apaga el indicador API	⚠ Pantalla de error con Reintentar (CP-05)	✓ Modo offline desde el service worker (CP-14)
App del wearable	—	✓ “Wearable desconectado”, conserva los últimos valores (CP-09)	✓ Sin efecto

Resultado: ✓ Verificado apagando cada pieza. **El wearable es completamente independiente:** no necesita al teléfono para nada, porque consulta la API por su cuenta y sólo usa el hub para publicar. Con el teléfono muerto (`pid` confirmado como inexistente) el reloj siguió generando durante 40 s sin inmutarse.

Ninguna caída provoca el cierre inesperado de otra app. Lo único que se pierde en cada caso es la función concreta que dependía de la pieza apagada.

Pruebas de validación de entrada

Además de los 19 casos, se comprobó que el hub descarta lo que no cumple el contrato:

Entrada	Respuesta
no-soy-json	400 {"error":"json inválido"}
{"nope":1}	400 {"error":"se esperaba {slug, category}"}
{"slug":"x","category":"y"}	200 con el estado actualizado

Evidencias

Archivo	Qué muestra
01-wearable-detenido.png	Wearable enlazado (GATT verde), sin generar
02-wearable-generando.png	Wearable generando: tema TECH, contadores vivos
03-telefono-feed.png	Feed de noticias del teléfono
04-telefono-wearable-conectado-alerta.png	Métricas en vivo + alerta de umbral
05-telefono-detalle-sincronizado.png	Detalle con aviso de que está en la TV
06-telefono-wearable-desconectado.png	Estado desconectado sin cierre inesperado
07-tv-grid-1920x1080.png	Smart TV con el grid 2x2 a resolución real
08-telefono-error-de-red.png	Manejo del error de red
09-telefono-apk-release-firmado.png	APK de release firmado, instalado y funcionando

Alumno: Luis Abraham Camacho Durán **Fecha:** 03 de agosto de 2026

Firma: 